



MobilityAPI

An OGC API – Moving Features server over MobilityDB

A thin compiled tier over MobilityDB · canonical Danish AIS example

net/http + pgxpool · single static binary · no MEOS in the tier



A Thin Tier over MobilityDB, Fat Database

- ▶ **OGC-compliant:** implements OGC API – Moving Features – Part 1: Core (collections, items, temporal geometry, derived queries, temporal properties)
- ▶ **Thin compiled tier:** net/http and a pgx connection pool; a single static binary, no cgo and no MEOS in the process
- ▶ **Fat database:** every temporal computation stays in MobilityDB; the tier never parses or transforms a trajectory in memory
- ▶ **REST for any client:** browsers, mobile, microservices and ETL consume mobility data without SQL or the PostgreSQL wire protocol
- ▶ **Bounded as data grows:** streaming responses, keyset pagination and index-using filters keep memory and paging bounded as collections grow

REQUEST FLOW

Browser

Mobile

ETL / lake

▼ HTTP

OGC API – Moving Features · MF-JSON

MobilityAPI (Go)

net/http + pgxpool

▼ SQL

PostgreSQL wire · pgx

MobilityDB / PostgreSQL

tgeompoint · STBOX · TSTZSPAN

Where the Work Happens

In the tier (Go)

- ▶ **Routing:** net/http with the Go 1.22 pattern mux; one handler per OGC resource
- ▶ **Pooling:** a pgx pool multiplexes requests onto a bounded set of database connections
- ▶ **Adapter:** crs URN to EPSG and interpolation Stepwise to Step, roughly fifty lines
- ▶ **Streaming:** responses are written row by row from a server-side cursor

In the database (MobilityDB)

- ▶ **Reads:** asMFJSON assembles MovingFeaturesJSON in SQL; the tier streams the text
- ▶ **Writes:** tgeompointFromMFJSON parses the payload into a tgeompoint
- ▶ **Filters:** bbox and datetime push to the GiST index; atTime clips
- ▶ **Measures:** speed, cumulativeLength and azimuth compute the derived properties

No MEOS in the tier

- ▶ The trajectory is never materialised or parsed in the tier
- ▶ It travels as MF-JSON text on the way out, MovingFeaturesJSON on the way in
- ▶ MobilityDB does the parsing, the geometry algebra and the temporal computations
- ▶ A single static binary, no cgo dependency

MobilityDB SQL vs MobilityAPI REST

PostgreSQL / MobilityDB

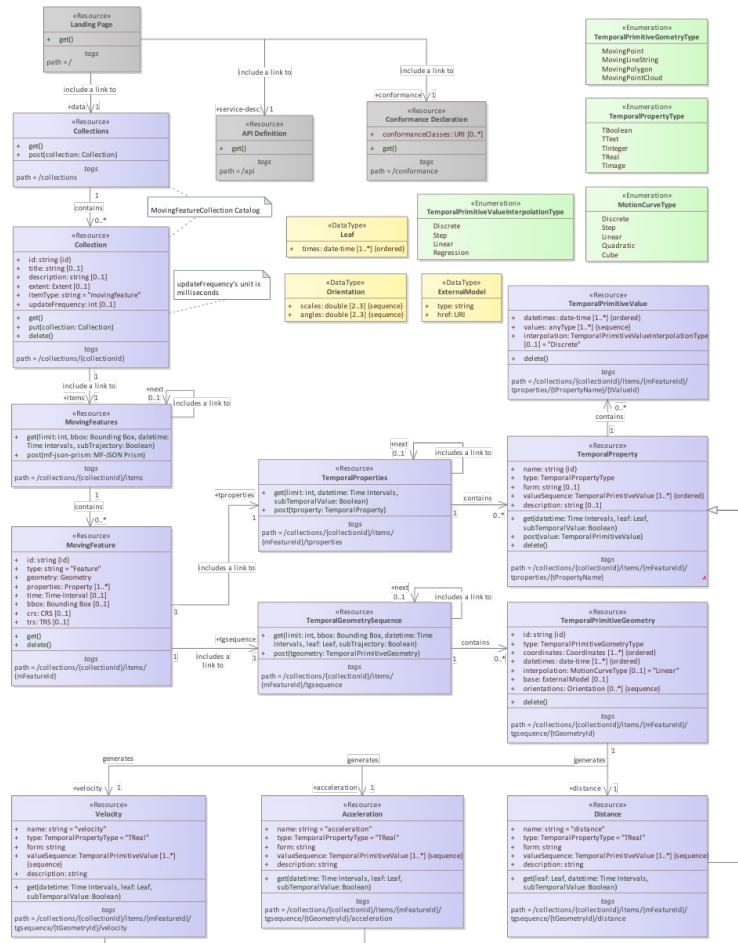
```
INSERT INTO ships (id, mmsi, name, trip)
VALUES (205106000, 205106000, 'MARCUS',
       tgeompoint '[
         Point(645829 6058110)@2026-02-26 06:00+00,
         Point(651280 6058390)@2026-02-26 06:25+00,
         Point(656731 6058670)@2026-02-26 06:50+00]'
       );
```

MobilityAPI

```
POST /collections/ships/items
Content-Type: application/json

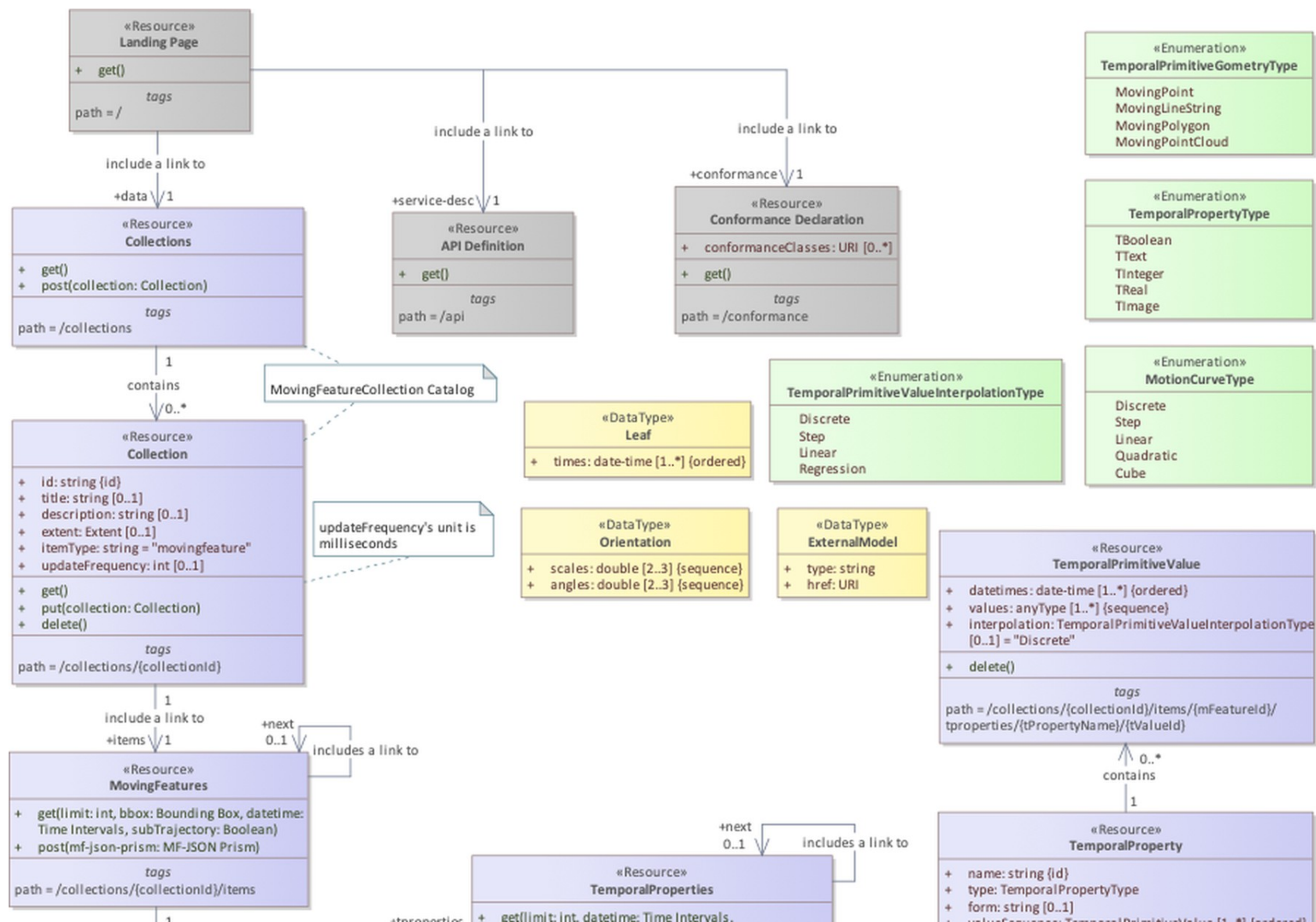
{
  "type": "Feature",
  "id": "205106000",
  "properties": { "name": "MARCUS" },
  "temporalGeometry": {
    "type": "MovingPoint",
    "datetimes": ["2026-02-26T06:00:00+00", ...],
    "coordinates": [[645829, 6058110], ...],
    "interpolation": "Linear"
  }
}
```

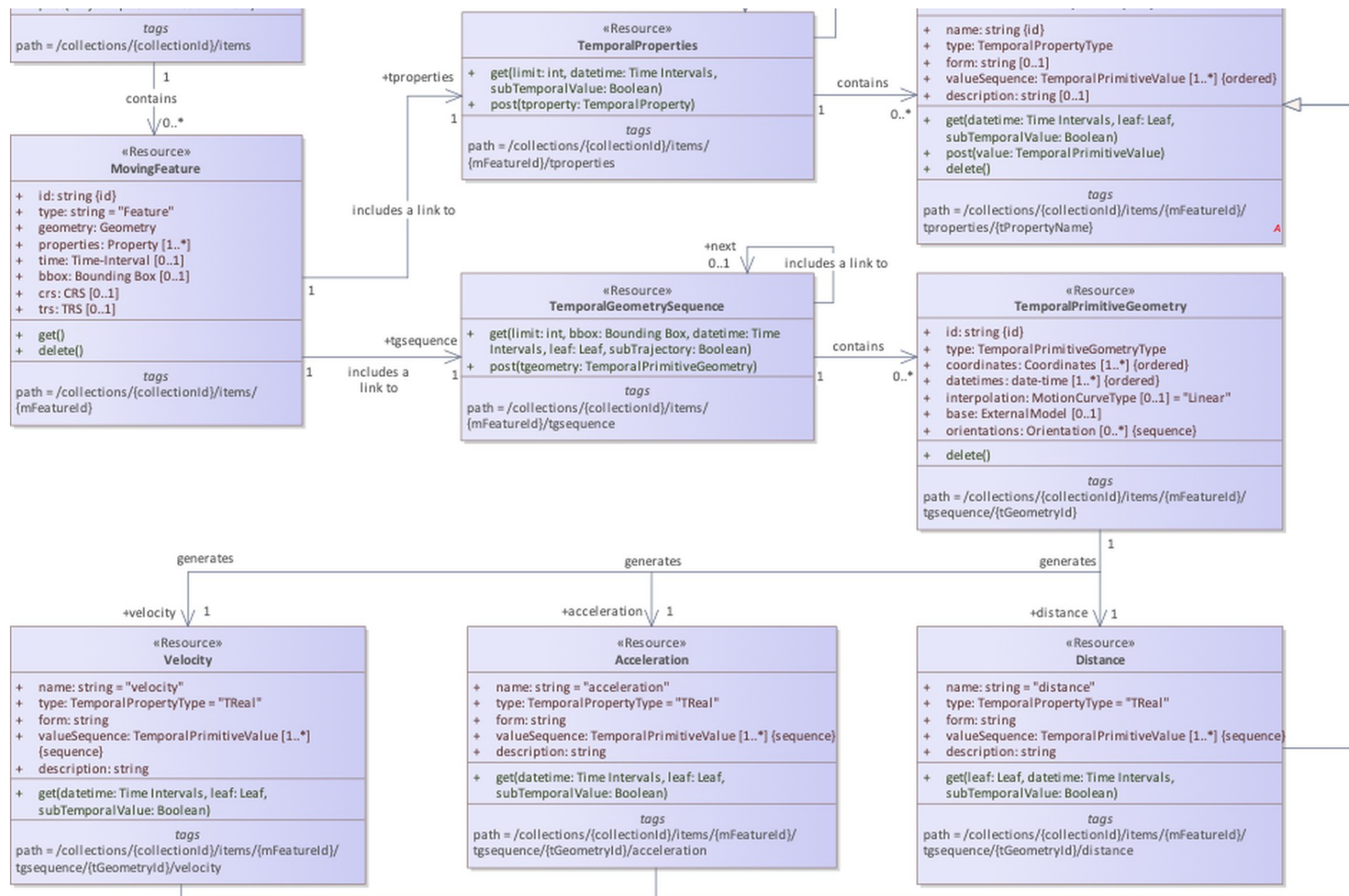
OGC API - Moving Features Data Model



Class diagram — OGC API - Moving Features - Part 1: Core (fig. 1)

- **Collection:** a named group of moving features (itemType movingfeature)
- **MovingFeature:** an item with id, properties, crs and trs
- **TemporalGeometry:** a MovingPoint with parallel coordinates and datetimes plus interpolation
- **Derived queries:** distance and velocity over a temporal geometry
- **TemporalProperties:** time-varying attributes derived from the trajectory





How MobilityAPI Persists Moving Features

STORED COLUMNS

collections

PK id	TEXT
title	TEXT
description	TEXT
itemType	TEXT
crs	INTEGER

ships

PK id	INTEGER
mmsi	BIGINT
name	TEXT
trip	tgeompoint

one feature table per collection

STANDARD FIELDS SERVED

Collection

- ▶ **Stored:** id, title, description, itemType, crs
- ▶ **Derived:** extent (spatial bbox and time interval)
- ▶ **Generated:** links

MovingFeature

- ▶ **Stored:** id, properties (mmsi, name)
- ▶ **Derived from trip:** temporalGeometry, bbox, time, geometry
- ▶ **From the registry:** crs
- ▶ **Constant:** trs
- ▶ **Generated:** links

How fields are provided

- ▶ Derived fields come O(1) from the tgeompoint and its inline STBOX
- ▶ Links are generated per request

A Day of Danish AIS Ship Traffic

- ▶ **Source:** Danish Maritime Authority AIS feed (aisdata.ais.dk)
- ▶ **Window:** 2026-02-26 to 2026-02-27
- ▶ **2,833 trajectories:** 1,878 distinct vessels (MMSI), each a MovingPoint
- ▶ **6.2 million instants:** observed position across the collection
- ▶ **CRS:** EPSG:25832 (ETRS89 / UTM 32N, metres)



Danish AIS vessel trajectories, 2026-02-26

The OGC Endpoint Surface

GET	POST	/collections
GET		/collections/{collectionId}
GET	POST	/collections/{collectionId}/items
GET	DELETE	/collections/{collectionId}/items/{mFeatureId}
GET	POST	/collections/{collectionId}/items/{mFeatureId}/tgsequence
GET		/collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/distance
GET		/collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/velocity
GET		/collections/{collectionId}/items/{mFeatureId}/tproperties
GET		/collections/{collectionId}/items/{mFeatureId}/tproperties/{tPropertyName}
GET		/ · /api · /conformance

Notes

- ▶ Every path above is OGC API – Moving Features – Part 1: Core; acceleration returns 501 (not derivable)
- ▶ Items query parameters: bbox, datetime, subtrajectory, leaf, limit, page cursor

WALKTHROUGH

The Ships Collection, End to End

List → filter → retrieve → derive → write, all over HTTP

List Collections · Retrieve One

GET /collections

200 OK

```
{
  "collections": [
    {
      "id": "ships",
      "title": "ships",
      "itemType": "movingfeature",
      "description": "Danish AIS vessel trajectories",
      "links": [
        { "rel": "items",
          "href": "/collections/ships/items" },
        { "rel": "enclosure",
          "href": "/collections/ships/export",
          "type": "application/x-ndjson" }
      ]
    }
  ]
}
```

GET /collections/ships

200 OK

```
{
  "id": "ships", "title": "ships",
  "itemType": "movingfeature",
  "description": "Danish AIS vessel trajectories",
  "crs": ["../def/crs/EPSG/0/25832"],
  "extent": {
    "spatial": { "bbox": [[152625, 5942006,
                           1020003, 6545239]] },
    "temporal": { "interval": [["2026-02-26 ...",
                                "2026-02-27 ..."]] } },
  "links": [ { "rel": "self" },
              { "rel": "items" },
              { "rel": "enclosure" } ]
}
```

- ▶ extent is the spatial bbox and time interval of the collection
- ▶ crs and links round out the standard collection resource

Stream the FeatureCollection, Page by Keyset

GET /collections/ships/items?limit=2

```
200 OK Content-Type: application/json
{
  "type": "FeatureCollection",
  "features": [
    { "id": "1", "type": "Feature",
      "properties": { "mmsi": 111219510, "name": "vessel 111219510" },
      "crs": { "type": "Name", "properties": {
        "name": "urn:ogc:def:crs:EPSG::25832" } },
      "bbox": [550889, 6328752, 616716, 6346258],
      "time": ["2026-02-26 13:28:08+01", "2026-02-26 14:41:04+01"],
      "temporalGeometry": { "type": "MovingPoint",
        "sequences": [ { "datetimes": [...], "coordinates": [...] } ],
        "interpolation": "Linear" },
      "links": [ { "rel": "self", "href": ".../items/1" } ] },
    { "id": "2", "type": "Feature", ... }
  ],
  "numberReturned": 2,
  "links": [ { "rel": "next",
    "href": "/collections/ships/items?after=2&limit=2" } ]
}
```

- ▶ **Streamed:** each Feature is written from a server-side cursor; memory is bounded for any page size
- ▶ **Keyset next:** the next link advances by id (after=), with no OFFSET
- ▶ **asMFJSON:** the temporalGeometry is assembled in SQL and streamed as text
- ▶ **limit:** caps the page; the tier enforces a maximum

Retrieve a Single Vessel

GET /collections/ships/items/1

200 OK

```
{
  "id": "1", "type": "Feature",
  "properties": { "mmsi": 111219510, "name": "vessel 111219510" },
  "crs": { "type": "Name", "properties": {
    "name": "urn:ogc:def:crs:EPSG::25832" } },
  "trs": { "type": "Link",
    "properties": { "type": "ogcdef",
      "href": ".../uom/ISO-8601/0/Gregorian" } },
  "bbox": [550889, 6328752, 616716, 6346258],
  "time": ["2026-02-26 13:28:08+01", "2026-02-26 14:41:04+01"],
  "geometry": { "type": "MultiLineString", "coordinates": [...] },
  "temporalGeometry": {
    "type": "MovingPoint",
    "sequences": [ { "datetimes": [...], "coordinates": [...] } ],
    "interpolation": "Linear" },
  "links": [ { "rel": "self",
    "href": "/collections/ships/items/1" } ]
}
```

- **One movement:** the full trajectory of one vessel as MovingFeaturesJSON
- **crs:** carried as an OGC URN; the tier maps to and from the EPSG form
- **interpolation:** Linear, so the vessel moves continuously between fixes
- **sequences:** one per continuous run; gaps split the trajectory

Vessels Intersecting a Bounding Box

```
GET /collections/ships/items?  
bbox=720000,6160000,738000,6178000
```

```
GET /collections/ships/items  
?bbox=720000,6160000,738000,6178000  
&limit=10000  
  
// bbox = minx,miny,maxx,maxy in the storage CRS  
// EPSG:25832 (metres)  
// 170 of 2,833 trajectories cross the box
```

- **bbox:** keeps vessels whose trajectory intersects the envelope
- **Index:** pushed to the MobilityDB GiST index via the overlap operator
- **CRS:** coordinates are in the collection's storage CRS (EPSG:25832)
- **Composable:** datetime, subtrajectory, limit and after



Trajectories crossing the box (teal) over other traffic (grey); the box in red

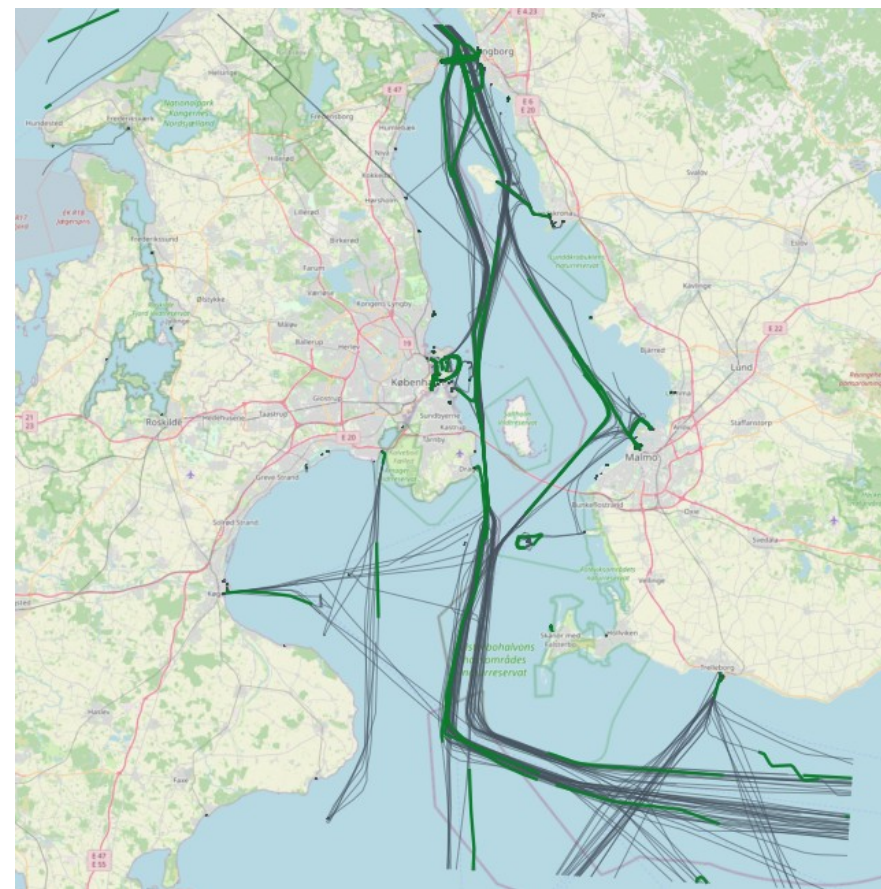
Restrict Trajectories to a Time Interval

GET /collections/ships/items?subtrajectory=true

```
GET /collections/ships/items
  ?subtrajectory=true
  &datetime=2026-02-26T14:00:00+01
    /2026-02-26T15:00:00+01

// datetime is an RFC 3339 interval (start/end)
// subtrajectory=true clips each trajectory to the
// window with atTime, instead of filtering trips
```

- ▶ **subtrajectory:** clips each trajectory to the datetime interval via atTime
- ▶ **datetime:** RFC 3339 interval; a bare instant is also accepted
- ▶ **Gap aware:** rows whose values fall entirely in a temporal gap are dropped
- ▶ **Validation:** subtrajectory requires a bounded interval



Subtrajectories within a one-hour window (green) over full trajectories (grey)

Create · Append · Delete

POST /collections/ships/items

```
{ "type": "Feature", "id": "900001",
  "properties": { "name": "demo" },
  "temporalGeometry": {
    "type": "MovingPoint", "interpolation": "Linear",
    "datetimes": ["2026-02-26T10:00:00+01", ...],
    "coordinates": [[600000, 6330000], ...] } }
```

```
201 { "message": "created", "id": "900001" }
```

POST /collections/ships/items/900001/tgsequence

```
// append a temporally-disjoint sub-trajectory
200 { "message": "appended", "id": "900001" }

// overlapping the existing time span:
409 { "code": "409",
      "description": "the sub-trajectory overlaps
                     the existing temporal geometry in time" }
```

DELETE /collections/ships/items/900001

204 remove the feature

- **merge:** append uses MobilityDB merge, which rejects time overlap; the tier maps that to 409
- **Derived measures:** velocity and distance are computed from the geometry and are read-only; change the geometry to change them
- **Member delete:** DELETE /.../tgsequence/{id} drops one geometry member by index via deleteTime; DELETE on the item removes the whole feature

The Temporal-Geometry Sequence

GET /collections/ships/items/1/tgsequence

```
200 OK
{
  "type": "TemporalGeometrySequence",
  "crs": { "type": "Name", "properties": {
    "name": "urn:ogc:def:crs:EPSG::25832" } },
  "geometrySequence": [
    { "id": 1, "type": "MovingPoint", "datetimes":
      ["2026-02-26T13:28:08+01", ...],
      "coordinates": [[721000.4, 6178001.2], ...],
      "lower_inc": true, "upper_inc": true }
  ],
  "interpolation": "Linear"
}
```

Selectors on this resource

- ▶ **datetime:** clip the sequence to an interval (atTime)
- ▶ **leaf:** return the geometry at one or more instants
- ▶ **POST:** append a temporally-disjoint sub-trajectory

Temporal geometry

- ▶ The temporal geometry is the trajectory itself
- ▶ Returned as MovingFeaturesJSON by asMFJSON

Distance · Velocity · Acceleration

GET /collections/ships/items/1/tgsequence/1/velocity

```
200 OK
{ "name": "velocity", "type": "TReal", "form": "m/s",
  "description": "Speed over ground ...",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01", ...],
      "values": [22.78, 11.03, ...],
      "interpolation": "Stepwise" } ],
  "links": [ { "rel": "self", ... } ] }
```

- ▶ **velocity:** speed over ground from speed(trip), piecewise-constant (Stepwise)
- ▶ **distance:** cumulative length from cumulativeLength(trip) (Linear)
- ▶ **acceleration:** not fabricated, because with linear position the speed is a step function whose derivative is not a finite function
- ▶ **Exact:** each measure is a MobilityDB function reshaped into an OGC temporalProperty (TReal)

GET /collections/ships/items/1/tgsequence/1/acceleration

```
501
{ "code": "501", "description":
  "acceleration is not derivable: linearly
  interpolated position gives a piecewise-constant
  speed, whose derivative is zero within each
  segment and undefined at the vertices" }
```

Time-Varying Attributes per Feature

GET /collections/ships/items/1/tproperties

```
200 OK
{
  "temporalProperties": [
    { "name": "fuel", "type": "TReal", "form": "L" },
    { "name": "cargo_temp", "type": "TReal", "form": "Cel" },
    { "name": "load", "type": "TReal", "form": "t" }
  ],
  "numberReturned": 3,
  "links": [ { "rel": "self", ... } ]
}
```

GET /collections/ships/items/1/tproperties/fuel?
leaf=...

```
{ "name": "fuel", "type": "TReal", "form": "L",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01"],
      "values": [22.78],
      "interpolation": "Discrete" } ] }
```

Property type

TBoolean · TText · TInteger · TReal

- ▶ **Stored:** user-supplied attributes — fuel, cargo temperature, load — held as native MobilityDB temporal values
- ▶ **Writable:** POST creates a property, POST appends values via merge, DELETE removes it
- ▶ **valueSequence:** datetimes and values with each segment's true interpolation
- ▶ **leaf / datetime:** leaf selects values at given instants (Discrete); datetime clips to an interval

Declared Classes, API Definition, Errors

Conformance classes

- ▶ movingfeatures / conf / common
- ▶ / conf / mf-collection
- ▶ / conf / movingfeatures
- ▶ ogcapi-common-1 / conf / core
- ▶ ogcapi-common-2 / conf / collections

API definition

- ▶ **/api:** an OpenAPI 3.0 definition
- ▶ **service-desc:** linked from the landing page
- ▶ **Landing:** self, service-desc, conformance, data
- ▶ Clients discover the surface at runtime

OGC error responses

- ▶ **400:** invalid parameters or body
- ▶ **404:** collection / feature not found
- ▶ **409:** sub-trajectory overlaps in time
- ▶ **501:** measure not derivable / derived write
- ▶ Body: { "code", "description" }

Streaming Keeps Memory Bounded

393 MB

in one streamed response

≈18 MB

resident, bounded by the stream not the result size

STANDARD ITEMS RESOURCE

- ▶ **Streaming:** the FeatureCollection is written row by row from a cursor, so memory is bounded for any size
- ▶ **Keyset pagination:** next links advance by id with no OFFSET, constant cost through large collections
- ▶ **Index-using filters:** bbox and datetime push to the MobilityDB GiST index; subtrajectory clips with atTime

Why it scales

Streaming and keyset pagination keep memory bounded for any collection size, a property of the standard items resource and not of the result size

Opt-in Capabilities Beyond the Standard

- ▶ **POST /.../bulk** bulk-insert a batch of (vehicleId, position, time, ...) observations in one request
- ▶ **GET /.../export** lakehouse feed: NDJSON or GeoParquet with trajectory WKB and a bbox/time sidecar
- ▶ **PUT /.../items/{id}** in-place replace of a feature
- ▶ **GeoJSON and GeoParquet** accepted in and out, alongside the MF-JSON default
- ▶ **gzip / deflate / br / zstd** compressed request and response bodies
- ▶ **POST / PUT / DELETE /collections** create, update and drop a collection and its feature table
- ▶ **POST / DELETE /.../tproperties** store and remove user-supplied TReal / TInt / TText / TBool attributes

Bulk Ingest

POST /collections/buses/bulk

Every minute a city posts each bus's position as a point and timestamp; one request appends them all.

```
POST /collections/buses/bulk          // every bus, one minute
Content-Encoding: gzip

[ { "id": "bus_42", "point": [4.3517, 50.8466], "t":
  "...T10:00:00Z" },
  { "id": "bus_57", "point": [4.3490, 50.8501], "t":
  "...T10:00:00Z" },
  ... 812 vehicles ... ]

201 { "observations": 812, "featuresCreated": 12,
      "featuresExtended": 800 }
```

- ▶ Unknown ids create a moving feature; out-of-order overlaps return 409
- ▶ GeoParquet ingest mirrors GET /export, so one file can backfill a whole collection

ENCODING

- ▶ **Formats** the body is GeoJSON or GeoParquet (MF-JSON default), the same records either way
- ▶ **Compression** gzip / deflate / br / zstd, decoded before parsing
- ▶ **Append** each record becomes one instant via appendInstant; the batch is atomic

Continuous Queries over Moving Features

- ▶ **Stream Extension:** a continuous query subscribes to a moving feature's temporal property or position
- ▶ **Register → stream:** the query returns a cquery link; results arrive as Server-Sent Events, one instant at a time
- ▶ **Transform:** a lifted scalar operation (ln, exp, ×, +) applied to a streaming tfloat — pointwise and exact
- ▶ **Windowed aggregate:** AVG · SUM · MIN · MAX · COUNT over COUNT, TUMBLING or HOPPING windows
- ▶ **StreamEngine seam:** in-process MEOS (cgo), Apache Flink or Kafka — the same cquery link, SSE stream and lifecycle

Animated Fleet over Live AIS

- ▶ **Shared data:** the same one day of Danish AIS as the static tutorial, loaded once as the ships collection
- ▶ **Streamed positions:** each vessel's WGS84 track is delivered as SSE; MEOS.js (WebAssembly) interpolates it in the browser
- ▶ **Live speed:** every dot is coloured by its windowed-average speed over ground (AIS SOG), accumulating as it streams
- ▶ **Growing chart:** a per-vessel speed chart grows with each observation — the colours of the MEOS.js example, live
- ▶ **Thin tier:** DeckGL and MapLibre render the fleet; the tier streams only stored positions, the browser does the temporal work

One Tier, Many MEOS Engines

Thin OGC API - Moving Features tier

MF-JSON / WKB wire format · streaming · keyset paging · index pushdown · no MEOS in the tier

▼ serves any MEOS-backed engine; the engine becomes a configuration choice

MobilityDB

PostgreSQL

Transactional serving over a live database

Operational

MobilityDuck

DuckDB

Embedded, Parquet-native lakehouse queries

Operational

MobilitySpark

Apache Spark

Distributed batch over very large histories

Operational

MobilityFlink

Apache Flink

Streaming, real-time moving features

Planned

shared MEOS types, the MF-JSON / WKB wire format, and a canonical named-function SQL dialect; the same tier fronts transactional databases, streaming engines, and lakehouses

MobilityDB · PostgreSQL

Transactional serving over a live MobilityDB database

The reference OGC output every other engine is matched against

▼ the engine is a configuration choice — a DSN scheme and a build tag select it

Connector

pgx pool

Live server connection;
pgx.ErrNoRows maps onto
the tier's ErrNoRows.

server

Build & deploy

static binary

Default build — static and
cgo-free; no engine libraries
linked.

default

SQL dialect

identity

Canonical named-function
SQL runs as-is; no name or
type translation.

as-is

Sweet spot

OLTP + index

Live transactional reads and
writes; GiST / SP-GiST index
pushdown.

live

The reference engine: the canonical MEOS named-function SQL served verbatim — MobilityDuck and MobilitySpark return this exact OGC output.

MobilityDuck · DuckDB

Embedded, Parquet-native analytics with no server

Identical OGC output to PostgreSQL over the same canonical SQL

▼ the engine is a configuration choice — a DSN scheme and a build tag select it

Connector

go-duckdb

Embedded and in-process via database/sql; selected by a duckdb:// DSN.

embedded

Build & deploy

-tags duckdb

MobilityDuck and spatial extensions LOAded per connection.

LOAD

SQL dialect

identity

Same canonical SQL as PostgreSQL; trajectory GeoJSON cast to text for a uniform scan.

as-is

Sweet spot

lakehouse

Parquet-native analytical queries over local files; no database server.

Parquet

Same thin tier, same canonical SQL, same Go response assembly — only the connector changes; identical OGC output to PostgreSQL.

MobilitySpark · Apache Spark

Distributed batch over very large movement histories

The same canonical SQL as PostgreSQL and DuckDB, over Spark Connect

▼ the engine is a configuration choice — a DSN scheme and a build tag select it

Connector

Spark Connect

Selected by a spark:// DSN; compiles against spark-connect-go.

Connect

Build & deploy

-tags spark

Placeholders inlined — Spark Connect's Sql takes no bind parameters.

no params

SQL dialect

identity

The canonical SQL names verbatim, like PostgreSQL and DuckDB; box accessors dispatch stbox vs tbox in one UDF via the WKB type tag, as-is

Sweet spot

distributed

Batch analytics over very large histories; th3index per-engine spatial indexing.

th3index

Same thin tier and Go assembly; MobilitySpark registers the canonical SQL names verbatim, so the only per-engine step is inlining the placeholders Spark Connect does not bind.

The Standard Surface, Recapped

COLLECTIONS

- ✓ list (GET)
- ✓ retrieve (GET)
- ✓ registry-backed

MOVING FEATURES

- ✓ insert (POST)
- ✓ retrieve / stream
- ✓ delete (DELETE)
- ✓ keyset paging

QUERYING

- ✓ bbox spatial filter
- ✓ datetime interval / instant
- ✓ subtrajectory clipping
- ✓ leaf selector

TEMPORAL GEOMETRY

- ✓ get sequence (GET)
- ✓ append sub-trajectory (POST)
- ✓ datetime / leaf clip
- ✓ member delete (DELETE)

DERIVED MEASURES

- ✓ distance
- ✓ velocity
- ✓ acceleration → 501 (not derivable)
- ✓ exact, as TReal

CONFORMANCE & SCALE

- ✓ 5 conformance classes
- ✓ OpenAPI at /api
- ✓ streaming + keyset
- ✓ index-using filters

Thanks for Listening

Questions?



Find Out More

- **OGC API - Moving Features - Part 1: Core**
<https://docs.ogc.org/is/22-003r3/22-003r3.html>
- **MobilityDB**
<https://github.com/MobilityDB/MobilityDB> · <https://mobilitydb.com>
- **MEOS / ecosystem overview**
<https://libmeos.org>
- **AIS data — Danish Maritime Authority**
<http://aisdata.ais.dk>
- **MobilityAPI tutorial notebook**
<https://github.com/MobilityDB/MobilityAPI/blob/master/tutorial/tutorial.ipynb>
- **MobilityAPI — Python implementation (PyMEOS)**
<https://github.com/MobilityDB/MobilityAPI-Python>
- **OGC MF API reference implementation — aistairc/mf-api**
<https://github.com/aistairc/mf-api>